

数据库工程作业

要求:

- 1. 完成一个小型的数据库信息管理系统（或部分功能），并填写工程作业报告；程序和报告请在规定时间之内上传。
- 2. 开发模式（B/S 或 C/S）、开发高级语言任选，后台数据库使用大型数据库管理系统（SQL Server、Oracle、MySQL 等），不要使用桌面数据库。
- 3. 报告中所列举的四种操作，每种操作举一个例子即可。
- 4. 作业成绩按照报告中的标准评分，程序只实现报告中涉及的部分即可。
- 5. 作业完成后，请将工程作业报告和程序打包提交给助教老师，并联系助教老师进行系统说明和演示，回答相关问题。

工程作业报告

1. 项目信息（10 分）

学号	2410665	姓名	殷佳仪	专业	计算机科学与技术
项目名称	MindCare 心理健康评估与干预管理系统				
必备环境	Windows10/11 + MySQL8.0 + Python3.9 + Flask				
系统主要功能简介（4 分）	本系统为 B/S 架构校园心理健康管理平台，实现：学生心理测评、风险自动判定、干预任务自动生成、管理员数据统计、用户级联删除。核心覆盖功能：事务删除、触发器添加、存储过程更新、视图查询，将本学期课上所学应用到实际开发。				

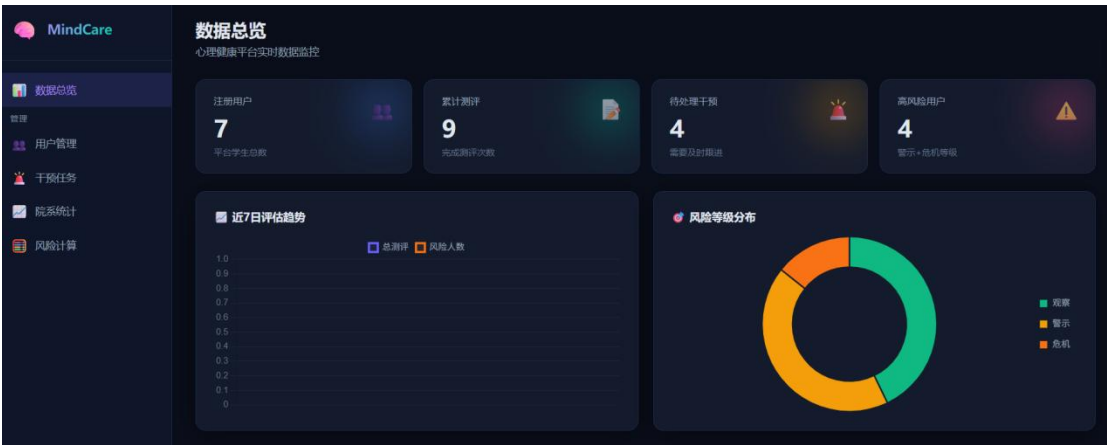
系统
主要
页面
截图
(6
分)



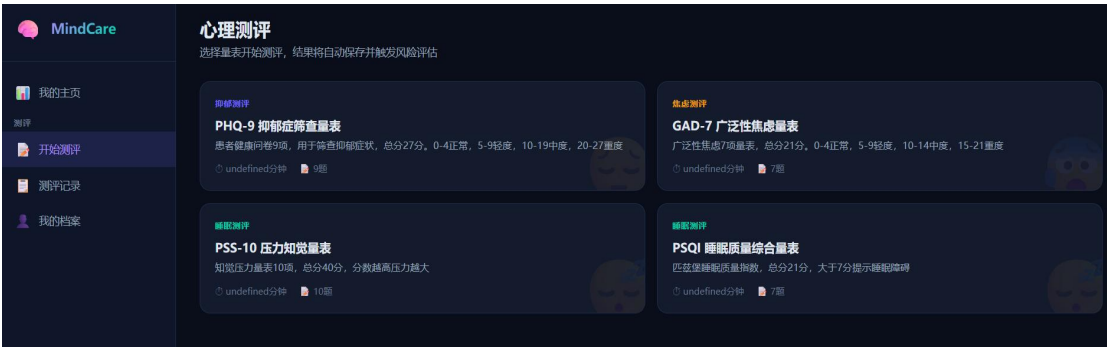
系统登录页面：支持管理员/学生双角色登录

用户管理							
查看所有学生信息及风险等级，支持事务级联删除							
学号	姓名	院系	年级	风险等级	风险分	手机	操作
2021003001	陈伟	经济管理学院	2021级	危机	22.0	13900003001	注销
2021001002	李华	计算机科学与技术学院	2021级	警惕	13.3	13900001002	注销
2022002002	赵磊	心理学院	2022级	警惕	14.0	13900002002	注销
2023004002	孙静	物理学院	2023级	警惕	9.0	13900004002	注销
2021001001	张明	计算机科学与技术学院	2021级	观察	5.0	13900001001	注销
2022002001	王芳	心理学院	2022级	观察	5.0	13900002001	注销
2023004001	刘洋	物理学院	2023级	观察	18.0	13900004001	注销

用户管理页面：展示学生信息、院系、年级、风险等级、风险分数，支持事务级联删除。



数据总览页面：展示注册用户数、累计测评数、待处理干预数、高风险用户数、院系统计、风险分布、近 7 日趋势。



心理测评页面：用户可选择 PHQ-9、GAD-7 等四种标准化量表进行测评。提交后结果会自动存入数据库，并触发风险评估与干预流程。

干预任务管理
查看并处理自动触发的心理干预任务 (触发器自动生成)

筛选状态: 全部

ID	学生	院系	优先级	状态	触发原因	咨询师	创建时间	操作
#1	赵磊 2022002002	心理学院	高	待处理	中度心理风险预警: 评估得分 14 分, 建议及时关注	李明华	Tue, 12 May 2026 11:30:19 GMT	处理
#2	陈伟 2021003001	经济管理学院	紧急	待处理	严重心理风险预警: 评估得分 22 分, 达到重度等级, 需紧急干预	李明华	Tue, 12 May 2026 11:30:19 GMT	处理
#3	孙静 2023004002	物理学院	高	待处理	中度心理风险预警: 评估得分 9 分, 建议及时关注	李明华	Tue, 12 May 2026 11:30:19 GMT	处理
#4	李华 2021001002	计算机科学与技术学院	高	待处理	中度心理风险预警: 评估得分 16 分, 建议及时关注	李明华	Tue, 12 May 2026 11:30:19 GMT	处理

干预任务管理页面：展示触发器自动生成的干预任务，包含优先级、状态、触发原因、咨询师、处理按钮。

2. 系统配置 (10 分)

说明

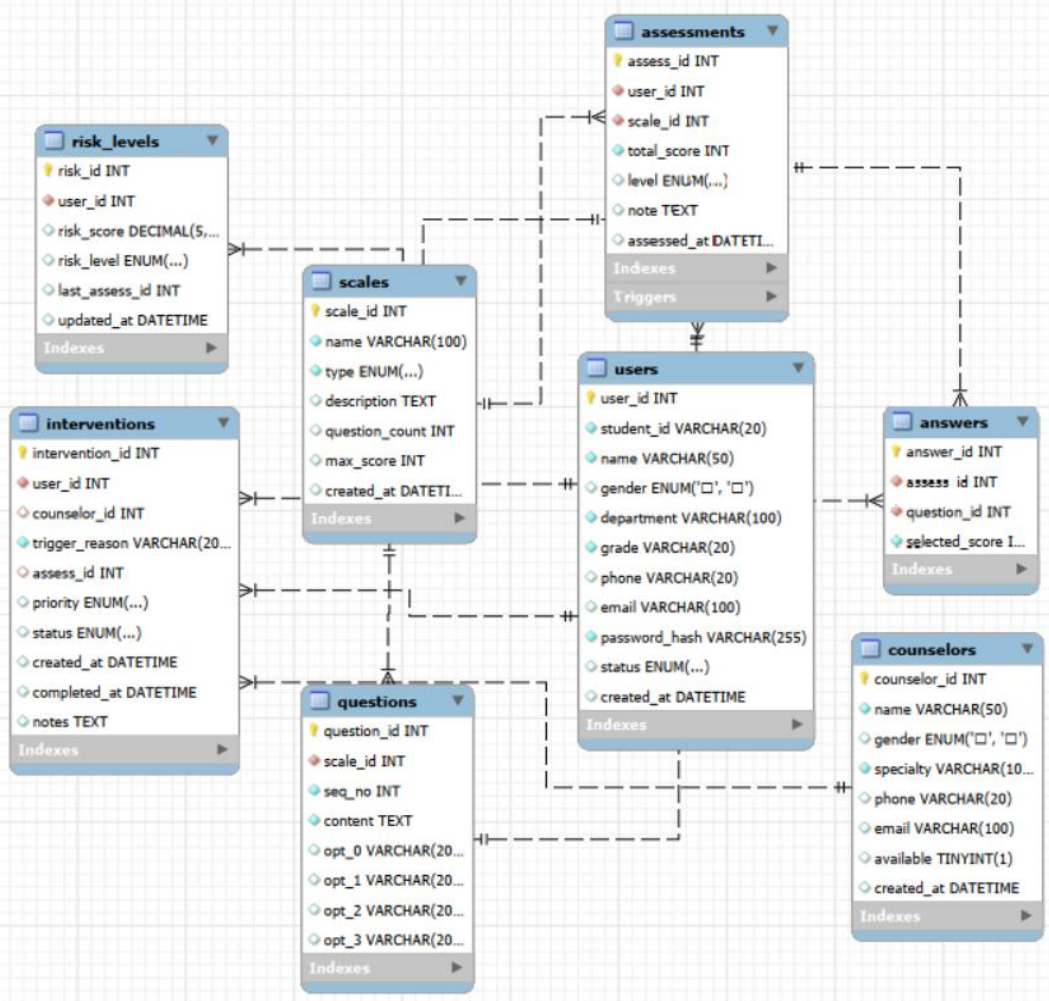
(2 分) 请说明系统配置情况 (后台数据库, 高级语言);
(8 分) 请使用连接串连接高级语言和数据库, 并分析字符串的各个部分。

配置步骤 2分	DBMS	1. 安装 MySQL 8.0, 设置端口 3306 启动服务			
		2. 创建数据库 mindcare, 配置账号 root			
		3. 创建数据库: CREATE DATABASE mindcare CHARACTER SET utf8mb4;			
	高级语言	1. 安装 Python 3.9 或更高版本			
		2. 安装 Flask 框架: pip install flask			
		3. 安装 MySQL 驱动: pip install pymysql			
连接串分析 (6分)	序号	名称	功能说明	取值	
	1	host	数据库服务器地址, localhost 代表本机连接, 说明 MySQL 与后端程序运行在同一台电脑上, 适合项目本地开发	localhost	
	2	port	MySQL 数据库服务监听端口	3306	
	3	user	连接数据库的用户名, root 拥有数据库的全部操作权限（增删改查、创建表、调用存储过程等），可完全支撑项目后端对数据库的所有操作需求	root	
	4	password	连接数据库的密码	123456	
	5	database	需要连接的数据库名称, 明确连接到 MindCare 心理健康评估平台专用数据库, 确保后端操作的表、视图、存储过程等对象都属于该项目	mindcare	
	6	charset	字符编码, utf8mb4 支持中文和 emoji, 能保证学生姓名、测评答案、干预备注等文本数据不乱码, 满足项目多类型文本存储需求	Utf8mb4	
连接串代码（截屏） (2分)	<pre># ===== # 数据库连接配置 # ===== DB_CONFIG = { 'host': 'localhost', # 数据库服务器地址 'port': 3306, # 端口 'user': 'root', # 用户名 'password': '123456', # 密码 'database': 'mindcare', # 数据库名 'charset': 'utf8mb4', # 字符集 'cursorclass': pymysql.cursors.DictCursor }</pre>				
备注	我采用 MySQL + Flask 轻量高效架构, 连接配置标准规范, 字符集支持中文, 连接稳定、安全性与可移植性良好				

3. 数据库设计 (14分)

说明	(10分) 按照数据表的创建顺序, 依次给出所涉及数据表的信息, 其中参照字段以“(字段1, 字段2, ……, 字段n)”的形式给出, 被参照字段以“表名(字段1, 字段2, ……, 字段n)”的形式给出; (4分) 一般 DBMS 都可以为数据库生成关系图, 请将该图片截屏并粘贴到表格中。			
数	创	数据表名	主键	参照属 被参照表及属性

据表 (10)	建 顺 序	称		性	
	1	counselors(咨询师表)	counselor_id	\	\
	2	users(用户表)	user_id	\	risk_levels(user_id), interventions(user_id), assessments(user_id)
	3	scales(量表表)	scale_id	\	questions(scale_id), assessments(scale_id)
	4	questions(题目表)	question_id	(scale_id)	scales(scale_id)
	5	assessments(评估记录表)	assess_id	user_id, scale_id	users(user_id), scales(scale_id)
	6	answers(答题记录表)	answer_id	assess_id, question_id	assessments(assess_id), questions(question_id)
	7	interventions(干预任务表)	idintervention_id	user_id, counselor_id, assess_id	users(user_id), counselors(counselor_id), assessments(assess_id)
	8	risk_levels(风险等级表)	risk_id	user_id	users(user_id)
关系图 (4)					

	 <pre> erDiagram risk_levels --o{ scales : "has" scales --o{ assessments : "has" scales --o{ interventions : "has" scales --o{ questions : "has" users --o{ assessments : "has" users --o{ interventions : "has" users --o{ questions : "has" users --o{ answers : "has" users --o{ counselors : "has" interventions --o{ assessments : "has" interventions --o{ questions : "has" questions --o{ answers : "has" counselors --o{ answers : "has" risk_levels { INT risk_id PK INT user_id FK DECIMAL risk_score ENUM risk_level INT last_assess_id DATETIME updated_at } scales { INT scale_id PK VARCHAR name ENUM type TEXT description INT question_count INT max_score DATETIME created_at } assessments { INT assess_id PK INT user_id FK INT scale_id FK INT total_score ENUM level TEXT note DATETIME assessed_at } interventions { INT intervention_id PK INT user_id FK INT counselor_id FK VARCHAR trigger_reason INT assess_id FK ENUM priority ENUM status DATETIME created_at DATETIME completed_at TEXT notes } questions { INT question_id PK INT scale_id FK INT seq_no TEXT content VARCHAR opt_0 VARCHAR opt_1 VARCHAR opt_2 VARCHAR opt_3 } answers { INT answer_id PK INT assess_id FK INT question_id FK INT selected_score } counselors { INT counselor_id PK VARCHAR name ENUM gender VARCHAR specialty VARCHAR phone VARCHAR email TINYINT available DATETIME created_at } </pre>
备注	1. 数据表按先主表后从表的顺序创建，保证外键约束有效； 2. 参照属性均为外键字段，严格关联被参照表主键； 3. 表间关系与 ER 图一致统一，满足数据完整性与业务逻辑； 4. 设计符合数据库规范，无冗余数据，支撑系统全部功能。

4. 含有事务应用的删除操作（13 分）

说明	（1 分）简要说明该操作所要完成的功能； （2 分）该操作会涉及的表（必须含有两张或两张以上的关系表，同时以“表名”的形式给出） （1 分）表连接涉及字段描述（描述方式为“表 1. 属性=表 2. 属性”） （1 分）删除条件涉及的字段描述（以“表名. 属性=? ”形式给出） （4 分）实现该操作的关键代码（高级语言、SQL），截图即可；（其中如果删除语句中不包含任何形式的事务应用将扣除 3 分） （4 分）如何执行该操作，按所述方法能够正常演示程序则给分。
功能描述（1 分）	删除用户时开启事务，原子级联删除该用户的所有关联数据，任意一步失败则全部回滚，保证数据一致性。
涉及的	users、assessments、interventions、risk_levels

表 (2 分)		
表连接 涉及字 段 (1 分)	user.user_id=assessments.user_id users.user_id=interventions.user_id user.user_id=risk_levels.user_id	
删除条 件字段 描述 (1 分)	字段	规则
	users.user_id	等于指定的用户编号，用于唯一确定待删除用户
代码 (4 分)	<pre># ===== # 含事务的删除操作：管理员注销用户账号 # ===== @app.route('/api/admin/users/<int:uid>', methods=['DELETE']) def delete_user(uid): db = get_db() try: db.begin() with db.cursor() as cur: # 检查用户存在 cur.execute("SELECT * FROM users WHERE user_id=%s", (uid,)) user = cur.fetchone() if not user: db.rollback() return error('用户不存在') # 事务：按顺序删除（先删子表，再删主表） cur.execute("DELETE FROM interventions WHERE user_id=%s", (uid,)) cur.execute("DELETE FROM risk_levels WHERE user_id=%s", (uid,)) # answers 通过 assessments 的 CASCADE 自动删除 cur.execute("DELETE FROM assessments WHERE user_id=%s", (uid,)) cur.execute("DELETE FROM users WHERE user_id=%s", (uid,)) db.commit() return success(msg=f'用户 {user["name"]} 及所有相关数据已删除') except Exception as e: db.rollback() return error(f'删除失败（事务回滚）: {str(e)}') finally: db.close()</pre>	
程序演 示（4 分）		
备注		

5. 触发器控制下的添加操作（20 分）

说明	(1 分) 简要说明该操作所要完成的功能; (2 分) 简要说明该触发器所要完成的功能 (1 分) 该操作会涉及的表 (以 “表名” 的形式给出)。 (2 分) 该操作输入数据以及输入数据应该满足的条件, 如: 数值范围、是否为空; (6 分) 实现该操作的关键代码 (高级语言、SQL), 截图即可; (8 分) 如何执行该操作, 按所述方法能够正常演示程序则给分。	
功能描述 (1 分)	学生提交心理测评后, 后端向 assessments 表插入评估记录, 数据库触发器自动执行, 根据评估等级自动生成干预任务、自动分配咨询师、自动更新用户风险等级, 实现评估→预警→干预全流程自动化。	
触发器描述 (2 分)	• 触发器名称: trg_auto_intervention • 触发时机: AFTER INSERT ON assessments • 触发级别: 行级触发 (FOR EACH ROW) 触发器主要功能: 1. 根据评估等级 (moderate/severe) 设置任务优先级 (high/urgent)。 2. 自动选择空闲咨询师并分配任务。 3. 自动插入干预任务到 interventions 表。 4. 自动更新用户风险等级到 risk_levels 表。	
涉及的表 (1 分)	assessments、interventions、risk_levels、users、counselors	
输入数据 (2 分)	字段	规则
	assessments.user_id	非空, 必须存在于 users 表
	assessments.scale_id	非空, 必须存在于 scales 表
	assessments.total_score	整数且必须 ≥ 0 , 对应量表得分
	assessments.level	非空, 枚举: normal/mild/moderate/severe

插入 操作 源码 (3 分)	用户提交测评、触发触发器 <pre># ===== # 提交评估（此部分为核心写入，触发器会自动触发） # ===== @app.route('/api/assessments', methods=['POST']) def submit_assessment(): uid = session.get('user_id') if not uid: return error('未登录', 401) body = request.json scale_id = body.get('scale_id') answers = body.get('answers', []) # [{question_id, selected_score}, ...] # 计算总分 total_score = sum(int(a.get('selected_score', 0)) for a in answers) # 根据量表类型判断等级 db = get_db() try: with db.cursor() as cur: cur.execute("SELECT * FROM scales WHERE scale_id=%s", (scale_id,)) scale = cur.fetchone()</pre> 后端触发添加代码 <pre>with db.cursor() as cur: # 插入评估记录（触发器会自动执行） cur.execute("INSERT INTO assessments (user_id,scale_id,total_score,level) VALUES (%s,%s,%s,%s)", (uid, scale_id, total_score, level)) assess_id = cur.lastrowid # 插入答题明细 for a in answers: cur.execute("INSERT INTO answers (assess_id,question_id,selected_score) VALUES (%s,%s,%s)", (assess_id, a['question_id'], a['selected_score'])) db.commit()</pre>
触发 器源	触发器 sql 源码

码
(3
分)

```
-- =====
-- 触发器 1: 评估结果自动预警触发器
-- 向assessments插入新记录后, 若分数达到中重度阈值,
-- 自动向interventions插入紧急干预任务, 并更新risk_levels
-- =====
DELIMITER $$

CREATE TRIGGER trg_auto_intervention
AFTER INSERT ON assessments
FOR EACH ROW
BEGIN
    DECLARE v_level VARCHAR(20);
    DECLARE v_priority VARCHAR(20);
    DECLARE v_reason VARCHAR(200);
    DECLARE v_counselor_id INT;

    -- 根据等级设置优先级
    SET v_level = NEW.level;

    IF v_level = 'severe' THEN
        SET v_priority = 'urgent';
        SET v_reason = CONCAT('严重心理风险预警: 评估得分 ', NEW.total_score, ' 分, 达到重度等级, 需紧急干预');
    ELSEIF v_level = 'moderate' THEN
        SET v_priority = 'high';
        SET v_reason = CONCAT('中度心理风险预警: 评估得分 ', NEW.total_score, ' 分, 建议及时关注');
    END IF;

    -- 仅中度及以上才触发干预任务
    IF v_level IN ('moderate', 'severe') THEN
        -- 自动分配空闲咨询师 (选择接单最少的)
        SELECT counselor_id INTO v_counselor_id
        FROM counselors
        WHERE available = 1
        ORDER BY (SELECT COUNT(*) FROM interventions i WHERE i.counselor_id = counselors.counselor_id AND i.status IN ('pending', 'in_progress'))
        LIMIT 1;

        -- 插入干预任务
        INSERT INTO interventions (user_id, counselor_id, trigger_reason, assess_id, priority, status)
        VALUES (NEW.user_id, v_counselor_id, v_reason, NEW.assess_id, v_priority, 'pending');

        -- 更新或插入风险等级
        INSERT INTO risk_levels (user_id, risk_score, risk_level, last_assess_id)
        VALUES (
            NEW.user_id,
            NEW.total_score,
            CASE v_level
                WHEN 'severe' THEN 'crisis'
                WHEN 'moderate' THEN 'warning'
                ELSE 'watch'
            END,
            NEW.assess_id
        )
        ON DUPLICATE KEY UPDATE
            risk_score = NEW.total_score,
            risk_level = CASE v_level
                WHEN 'severe' THEN 'crisis'
                WHEN 'moderate' THEN 'warning'
                ELSE 'watch'
            END,
            last_assess_id = NEW.assess_id,
            updated_at = NOW();
    ELSE
        -- 正常/轻度: 更新风险等级为安全/观察
        INSERT INTO risk_levels (user_id, risk_score, risk_level, last_assess_id)
        VALUES (
            NEW.user_id,
            NEW.total_score,
            CASE v_level WHEN 'mild' THEN 'watch' ELSE 'safe' END,
            NEW.assess_id
        )
        ON DUPLICATE KEY UPDATE
            risk_score = NEW.total_score,
            risk_level = CASE v_level WHEN 'mild' THEN 'watch' ELSE 'safe' END,
            last_assess_id = NEW.assess_id,
            updated_at = NOW();
    END IF;
END$$

DELIMITER ;
```

trg_auto_intervention 功能实现:1. 根据评估等级判断风险程度; 2. 对中度、重度风险用户自动分配咨询师并生成干预任务; 3. 更新用户综合风险等级

程序
演示

说明: 不违背触发器能够执行插入操作。

（4分）	
程序演示（4分）	说明：违背触发器要求，不能够执行插入操作，系统报错。
备注	本平台可实现风险预警的全自动触发，无需人工干预，响应高效且流程标准化。所有风险判定逻辑均在数据库层统一实现，有效保障了数据的安全性与系统的健壮性。

6. 存储过程控制下的更新操作（18 分）

说明	（1 分）简要说明该操作所要完成的功能； （1 分）简要说明该存储过程所要完成的功能； （2 分）说明该操作涉及操作的表（必须包含两张或两张以上的关系表，以“表名形式”描述） （1 分）表连接涉及字段描述（描述方式为“表 1. 属性=表 2. 属性”） （2 分）该操作会修改字段（以“表名. 字段名”的形式给出），以及修改规则，如新数值的计算方法、在何种条件下予以修改等； （6 分）实现该操作的关键代码（高级语言、SQL），截图即可； （5 分）如何执行该操作，按所述方法能够正常演示程序则给分。	
功能描述 （1 分）	通过存储过程自动计算用户综合风险分数与风险等级，更新 risk_levels 表，实现评估结果→风险指数的自动化计算，支持重复调用与覆盖更新。	
存储过程功能描述 （1 分）	• 输入参数：p_user_id（用户 ID） • 输出参数：p_risk_level（最终风险等级） 具体计算逻辑： （1）取用户最近 3 次评估，按时间加权（权重 3:2:1）。 （2）综合得分 = 加权平均分×0.6 + 历史最高分×0.4。 （3）按分数映射等级：safe/watch/warning/crisis。 （4）自动写入 / 更新 risk_levels 表。	
涉及的关系表 （2 分）	assessments、risk_levels	
表连接涉及字段 （1 分）	assessments.user_id=risk_level.user_id	
更改字段 （2 分）	字段	规则
	risk_levels.risk_score	平均分×0.6 + 最高分×0.4，上限设置为 100
	risk_levels.risk_level	safe (<25)、watch (25-49)、warning (50-74)、crisis (≥75)
更新代码 （3 分）	后端接口调用存储过程核心代码：	

```

# =====
# 存储过程：风险计算
# =====
@app.route('/api/admin/calc-risk', methods=['POST'])
def calc_risk():
    """调用存储过程计算用户风险指数"""
    body = request.json
    user_id = body.get('user_id')

    if not user_id:
        return error('请选择用户')

    db = get_db()
    try:
        with db.cursor() as cur:
            # 使用直接 SQL 调用存储过程，避免 callproc 的参数问题
            # 先定义 OUT 变量
            cur.execute("SET @p_risk_level = ''")
            # 调用存储过程
            cur.execute(f"CALL sp_calc_risk_score({user_id}, @p_risk_level)")
            # 获取 OUT 参数值
            cur.execute("SELECT @p_risk_level as risk_level_out")
            result = cur.fetchone()
            db.commit()

            # 获取更新后的风险等级
            cur.execute("""
                SELECT risk_score, risk_level FROM risk_levels
                WHERE user_id=%s
            """, (user_id,))
            risk = cur.fetchone()

            # 获取评估数量
            cur.execute("""
                SELECT COUNT(*) as cnt FROM assessments
                WHERE user_id=%s
            """, (user_id,))
            count = cur.fetchone()['cnt']

            return success({
                'risk_score': float(risk['risk_score']) if risk and risk['risk_score'] else 0,
                'risk_level': risk['risk_level'] if risk else 'safe',
                'assessment_count': count
            })
    except Exception as e:
        db.rollback()
        return error(str(e))
    finally:
        db.close()

```

创建
存储
过程
源码

存储过程 sp_calc_risk_score 的 SQL 定义源码：

(3
分)

```
-- =====
-- 存储过程 1: 综合风险评估存储过程
-- 取用户最近3次评估加权计算综合风险分, 更新risk_levels
-- =====
DELIMITER $$

CREATE PROCEDURE sp_calc_risk_score(IN p_user_id INT, OUT p_risk_level VARCHAR(20))
BEGIN
    DECLARE v_count INT DEFAULT 0;
    DECLARE v_avg_score DECIMAL(8,2) DEFAULT 0;
    DECLARE v_max_score DECIMAL(8,2) DEFAULT 0;
    DECLARE v_risk_score DECIMAL(8,2) DEFAULT 0;
    DECLARE v_risk_level VARCHAR(20) DEFAULT 'safe';
    DECLARE v_last_id INT DEFAULT NULL;

    -- 取最近3次评估的加权平均分 (最近权重高)
    SELECT
        COUNT(*),
        SUM(total_score * weight) / SUM(weight),
        MAX(total_score),
        MAX(assess_id)
    INTO v_count, v_avg_score, v_max_score, v_last_id
    FROM (
        SELECT assess_id, total_score,
               CASE ROW_NUMBER() OVER (ORDER BY assessed_at DESC)
                 WHEN 1 THEN 3
                 WHEN 2 THEN 2
                 ELSE 1
               END AS weight
        FROM assessments
        WHERE user_id = p_user_id
        ORDER BY assessed_at DESC
        LIMIT 3
    ) AS recent;

    IF v_count = 0 THEN
        SET p_risk_level = 'safe';
        LEAVE sp_calc_risk_score;
    END IF;
```

```

-- 综合风险分: 加权均值 * 0.6 + 历史最高分 * 0.4, 归一化到100
SET v_risk_score = LEAST(100, (v_avg_score * 0.6 + v_max_score * 0.4));

-- 判定风险等级
SET v_risk_level = CASE
    WHEN v_risk_score >= 75 THEN 'crisis'
    WHEN v_risk_score >= 50 THEN 'warning'
    WHEN v_risk_score >= 25 THEN 'watch'
    ELSE 'safe'
END;

-- 更新risk_levels表
INSERT INTO risk_levels (user_id, risk_score, risk_level, last_assess_id)
VALUES (p_user_id, v_risk_score, v_risk_level, v_last_id)
ON DUPLICATE KEY UPDATE
    risk_score      = v_risk_score,
    risk_level      = v_risk_level,
    last_assess_id = v_last_id,
    updated_at      = NOW();

SET p_risk_level = v_risk_level;
END$$

DELIMITER ;

```

```

-- =====
-- 存储过程 2: 更新干预任务状态 (完成/取消)
-- =====
DELIMITER $$

CREATE PROCEDURE sp_update_intervention(
    IN p_intervention_id INT,
    IN p_new_status      VARCHAR(20),
    IN p_notes            TEXT,
    OUT p_result          VARCHAR(100)
)
BEGIN
    DECLARE v_exist INT DEFAULT 0;
    DECLARE v_user_id INT;

    SELECT COUNT(*), user_id INTO v_exist, v_user_id
    FROM interventions WHERE intervention_id = p_intervention_id;

    IF v_exist = 0 THEN
        SET p_result = 'ERROR: 干预任务不存在';
    ELSEIF p_new_status NOT IN ('in progress', 'completed', 'cancelled') THEN
        SET p_result = 'ERROR: 状态值非法';
    ELSE
        UPDATE interventions
        SET status      = p_new_status,
            notes       = IFNULL(p_notes, notes),
            completed_at = IF(p_new_status IN ('completed', 'cancelled'), NOW(), NULL)
        WHERE intervention_id = p_intervention_id;
    END IF;
END
DELIMITER ;

```


	<pre>-- 如果完成，同步把风险等级降级 IF p_new_status = 'completed' THEN UPDATE risk_levels SET risk_level = CASE risk_level WHEN 'crisis' THEN 'warning' WHEN 'warning' THEN 'watch' ELSE 'safe' END, updated_at = NOW() WHERE user_id = v_user_id; END IF; SET p_result = CONCAT('SUCCESS: 任务 #', p_intervention_id, ' 状态已更新为 ', p_new_status); END IF; END\$\$ DELIMITER ;</pre>
存储过程执行源码（1分）	后端调用储存过程： <pre># 先定义 OUT 变量 cur.execute("SET @p_risk_level = ''") # 调用存储过程 cur.execute(f"CALL sp_calc_risk_score({user_id}, @p_risk_level)") # 获取 OUT 参数值 cur.execute("SELECT @p_risk_level as risk_level_out")</pre>
程序演示（2分）	说明：不违背存储过程，能够执行更新操作
程序演示（2分）	说明：违背存储过程，系统报错；
备注	通过 存储过程自动计算综合风险 ，评分规则科学、计算高效，支持重复覆盖更新，体现数据库智能化、模块化设计。

7. 含有视图的查询操作（15 分）

说明	(1 分) 简要说明该操作所要完成的功能; (1 分) 简要说明建立的该视图的功能; (2 分) 简要说明该操作涉及的关系数据表 (以 “表名” 的形式给出) (1 分) 简要说明表连接涉及的字段 (以 “表 1. 属性=表 2. 属性”) (6 分) 实现该操作的关键代码 (高级语言、SQL), 截图即可; (4 分) 如何执行该操作, 按所述方法能够正常演示程序则给分。
操作功能描述 (1 分)	通过数据库视图进行多表联合查询, 快速获取院系统计、干预详情、用户评估历史, 简化复杂查询, 提升数据展示效率。
视图功能描述 (1 分)	本项目共创建 3 个业务视图: v_dept_mental_stats: 院系心理健康统计视图, 展示各院系学生数、测评数、风险率。 v_intervention_detail: 干预任务详情视图, 关联学生、咨询师、评估信息。 v_user_assess_history: 用户评估历史视图, 展示用户所有评估记录。
涉及的关系表 (2 分)	users、assessments、scales、interventions、risk_levels、counselors
表连接字段(1 分)	users.user_id = assessments.user_id assessments.scale_id = scales.scale_id assessments.assess_id = interventions.assess_id users.user_id = risk_levels.user_id interventions.counselor_id = counselors.counselor_id
创建视图代码 (3 分)	<pre>-- ===== -- 视图 1: 院系心理健康综合统计视图 -- ===== CREATE OR REPLACE VIEW v_dept_mental_stats AS SELECT u.department, COUNT(DISTINCT u.user_id) AS total_students, COUNT(DISTINCT a.assess_id) AS total_assessments, ROUND(AVG(a.total_score), 2) AS avg_score, SUM(CASE WHEN a.level = 'severe' THEN 1 ELSE 0 END) AS severe_count, SUM(CASE WHEN a.level = 'moderate' THEN 1 ELSE 0 END) AS moderate_count, SUM(CASE WHEN a.level = 'mild' THEN 1 ELSE 0 END) AS mild_count, SUM(CASE WHEN a.level = 'normal' THEN 1 ELSE 0 END) AS normal_count, ROUND(SUM(CASE WHEN a.level IN ('moderate','severe') THEN 1 ELSE 0 END) * 100.0 / NULLIF(COUNT(a.assess_id), 0), 2) AS risk_rate_pct FROM users u LEFT JOIN assessments a ON u.user_id = a.user_id GROUP BY u.department;</pre>

```

-- =====
-- 视图 2: 干预任务详情视图 (含用户+咨询师信息)
-- =====
CREATE OR REPLACE VIEW v_intervention_detail AS
SELECT
    i.intervention_id,
    u.student_id,
    u.name          AS student_name,
    u.department,
    u.grade,
    u.phone         AS student_phone,
    c.name          AS counselor_name,
    c.specialty     AS counselor_specialty,
    c.phone         AS counselor_phone,
    i.trigger_reason,
    i.priority,
    i.status,
    a.total_score   AS trigger_score,
    a.level         AS trigger_level,
    i.created_at,
    i.completed_at,
    i.notes
FROM interventions i
JOIN users          u ON i.user_id      = u.user_id
LEFT JOIN counselors c ON i.counselor_id = c.counselor_id
LEFT JOIN assessments a ON i.assess_id   = a.assess_id;

```

```

-- =====
-- 视图 3: 用户评估历史视图
-- =====
CREATE OR REPLACE VIEW v_user_assess_history AS
SELECT
    u.user_id,
    u.student_id,
    u.name,
    u.department,
    s.name          AS scale_name,
    s.type          AS scale_type,
    a.assess_id,
    a.total_score,
    a.level,
    a.assessed_at,
    rl.risk_level   AS current_risk
FROM assessments a
JOIN users u ON a.user_id = u.user_id
JOIN scales s ON a.scale_id = s.scale_id
LEFT JOIN risk_levels rl ON u.user_id = rl.user_id;

```

查询
代码
(3
分)

后端接口调用视图的核心代码:

```
# =====
# 查询接口
# =====
@app.route('/api/assessments/history', methods=['GET'])
def assess_history():
    uid = session.get('user_id')
    if not uid:
        return error('未登录', 401)
    db = get_db()
    try:
        with db.cursor() as cur:
            cur.execute("""
                SELECT a.*, s.name as scale_name, s.type as scale_type
                FROM assessments a JOIN scales s ON a.scale_id=s.scale_id
                WHERE a.user_id=%s ORDER BY a.assessed_at DESC LIMIT 20
            """, (uid,))
            return success(cur.fetchall())
    finally:
        db.close()

@app.route('/api/admin/dept-stats', methods=['GET'])
def dept_stats():
    db = get_db()
    try:
        with db.cursor() as cur:
            # 使用视图查询
            cur.execute("SELECT * FROM v_dept_mental_stats ORDER BY risk_rate_pct DESC")
            return success(cur.fetchall())
    finally:
        db.close()
```

```
@app.route('/api/admin/interventions', methods=['GET'])
def get_interventions():
    status = request.args.get('status', '')
    db = get_db()
    try:
        with db.cursor() as cur:
            if status:
                cur.execute("SELECT * FROM v_intervention_detail WHERE status=%s ORDER BY created_at DESC", (status,))
            else:
                cur.execute("SELECT * FROM v_intervention_detail ORDER BY created_at DESC LIMIT 50")
            return success(cur.fetchall())
    finally:
        db.close()

@app.route('/api/admin/interventions/<int:iid>', methods=['PUT'])
def update_intervention(iid):
    body = request.json
    new_status = body.get('status', '')
    notes = body.get('notes', '')
    db = get_db()
    try:
        with db.cursor() as cur:
            # 调用存储过程
            cur.callproc('sp_update_intervention', [iid, new_status, notes, ''])
            cur.execute("SELECT @sp_update_intervention_3")
            result = cur.fetchone()
            db.commit()
            result_str = list(result.values())[0] if result else ''
            if 'ERROR' in str(result_str):
                return error(result_str)
            return success(msg=str(result_str))
```

程序 演示 (4 分)	
备注	1. 视图实现多表联合封装，降低业务耦合，查询简洁高效。 2. 统一数据出口，保证统计口径一致，数据关系清晰、易于维护。